

P2.9 gate record

Run: 7 October 2026, one continuous session on the deployed app **App:** `avianv1.streamlit.app`, from `amrxshariff/avian_v1` at the cut of private `0f2dd41` **Verdict:** **5 of 5 pass.**

Same discipline as Gates A to D: one continuous session, a written verdict per step, no restart. Run in a private window so no session state carried over.

1. Why this gate exists

P2.9 put the app on Streamlit Community Cloud from a public repository. Everything before this point was verified locally or by reasoning. Three things could only be established by a visitor using the deployed app:

- whether the project key reaches the app at all on Community Cloud (D-68)
- whether a stranger’s first run lands somewhere useful without a key or an upload
- whether the correction loop — the product’s core claim — works end to end on a machine that is not the author’s

2. The five steps

Numbered as the plan states them (`docs/p2_9_deployment_plan.md`, step 7).

#	Check	Verdict
1	First run, no key, no upload	Pass
2	Upload without a pasted key: classified, not keyless	Pass
3	Grey nodes: present, and correctable	Pass
4	Provenance line: shown for the demo and for an uploaded build	Pass
5	Zero crashes across the session	Pass

Step 1 — first run, no key, no upload

Lands on the welcome screen: the headline, the four-point explanation, the upload panel with the key field collapsed, the privacy line including the one-hour drop, and the demo offered below.

The plan’s criterion first read “lands on the demo”. The welcome screen with the demo offered is the designed first run, so the pass is recorded against what happens.

This is the first-run path added as **New** in the 22 September scope revision — a path that works with no key and no upload. It works.

Step 2 — upload without a pasted key

The D-68 verification. A ~305-person export, uploaded with no key pasted, was classified, not built keyless. The provenance line for the build read:

Classified by `claude-sonnet-5`, using this demo’s shared allowance.

“This demo’s shared allowance” is `provenance.py`’s wording for the project key. So this line is the direct evidence: the build ran on a key the visitor did not supply.

D-68 was closed as “not a defect as reported” on evidence covering `streamlit run` and the secrets file it reads, and its entry states plainly that Community Cloud itself was not verified and names this check as

the verification. It is now verified: a root-level `ANTHROPIC_API_KEY` in Community Cloud's Secrets reaches `os.environ`, `keys.py` finds it, and a visitor gets a classified network without pasting anything.

The free tier works on deployment, which is what the whole P2.9b key slice was for.

Step 3 — grey nodes: present, and correctable

Present. The demo loaded with 442 people, **124 needing review**, matching `data/demo/provenance.json` exactly: coloured nodes with grey ones among them.

Correctable. One person in the review queue was assigned a group. The needs-review count dropped, and the node took its colour.

This is the Phase 2 checkpoint condition — a deployed app that renders a real network and lets someone correct a grey node — and it is met.

Step 4 — provenance line: demo and uploaded build

The demo's line read:

Example network, classified in advance by `claude-sonnet-5`. No real person appears in it.

Both halves of `demo_line()` are there, including the `real_people == 0` clause, which before P2.9b existed only in a file nobody read. The uploaded build's line is the one quoted under step 2.

Step 5 — zero crashes

None across the session, which included two builds, chat questions, corrections and a session expiry.

3. Verified after the re-cut

Not gate steps. These were the first confirmation on the deployed app of fixes made after the gate ran, and they belong here because the gate is the only record of the deployed app's behaviour.

- **D-80** — the chat shows “N free questions left” before the first question. The allowance existed, was tested, and had never been wired to the panel.
- **D-82** — the title reads “Network Visualiser”. The web-editor rename lost to a force-push is restored.
- **Session expiry** — after an hour of inactivity the banner reads that the network was dropped “with your corrections, notes and API key”, and the welcome screen returns. `lifetime.py` clearing the whole session, as designed, observed in the wild.

4. What the gate surfaced

A gate's job is as much to find things as to pass.

D-80 — the chat allowance was never applied

Found during step 2's session. `chat_allowance.py` implemented §6.6 completely and its tests passed by calling it directly; `chat.py` never called `current()` or `record_question()`. Nothing was counted, nothing shown, and the eleventh question did not fail. On the project key the chat was unlimited at roughly \$0.02 a question.

Logged High. Investigating it found **D-81**: the spend meters had the same shape — `Meters`, `DailyLedger` and `GenerationMeter` were built, tested, and never constructed outside `spend.py`'s own tests. No \$5 per build, no \$30 per day, and `data/spend/daily.json` never written. Both are fixed.

D-72 — measured

~305 people classified in about **3 minutes**, spinner only, no progress and no recovery. The deploy log gives the supporting detail: encoding 305 profiles took **one second** (**Batches: 10/10 [00:01<00:00]**). The three minutes was the classifier essentially alone — the one stage that already stops at batch boundaries for the budget meter, and already knows its denominator from `provenance["uncached_titles"]`.

Worse here than locally, not because it is slower (about 0.6 s per person against about 0.7 locally) but because it is a stranger’s first impression on a shared container.

KeyError: 'src' — not a defect

At 15:58 on 7 October, during the force-push of a re-cut, the running app raised `KeyError: 'src' importing src.onet`. Streamlit was hot-reloading from the old tree while every file beneath it was replaced at once. It recovered unaided: new machine at 15:58:10, updated at 15:58:17, clean pull at 16:04:31.

Recorded so nobody debugs it next time: **a cut causes one failed rerun before the redeploy.**

Step 6’s qualification

After the `fileWatcherType = "none"` config, the five-minute periodic lines stopped. Occasional lines remained: four in the two hours after the redeploy, in pairs about three minutes apart.

Their source is not established. In Streamlit 1.41 the only emitter of “Examining the path of ...” is `LocalSourcesWatcher`, and `app_session.py` constructs that watcher only when `fileWatcherType` is not `"none"`. A session running with the new config cannot produce the line. So the remaining lines came from a process that did not have it yet, which is unverified. The honest statement is that the periodic noise is gone, not that the log is silent.

5. Conditions

- Run from one browser session, private window, no restart.
- The uploaded export at step 3 was a ~305-row test file, not the author’s full network.
- Steps 1 to 5 ran against the cut of `0f2dd41`, before D-80 and D-81 were fixed. The §3 verifications ran against the later cut. No step’s verdict depends on the difference.